



ASSET ALLOCATION PERFORMANCE FORECASTING

DATA CHALLENGE ENS

Authors: Víctor Camps Abarca
Saïd Tellez Isibasi
Steven Yu

NANTES, JAN 28TH 2026

Contents

1	Introduction	1
1.1	Challenge Context	1
1.2	Dataset Description	1
1.3	Problem Formulation	1
1.3.1	Mathematical Definition	1
1.3.2	Evaluation Metric	2
2	Data Analysis	2
2.1	Missing Values Analysis	2
2.1.1	Overview	2
2.1.2	Observations	2
2.1.3	Why Missing Values Are Problematic	3
2.1.4	Benchmark's Approach	3
3	Conclusion	4

List of Figures

List of Tables

1 Introduction

This task is part of a datachallenge on asset allocation performance forecasting. The challenge aims to predict whether a given asset allocation will have a positive or negative return on the following trading day.

1.1 Challenge Context

In systematic trading, traders are constantly faced with numerous candidate asset allocations - portfolio constructions based on recent signals, liquidity flows, or historical patterns. The central question is: *Can we predict whether a given asset allocation is worth following or shorting?*

An **asset allocation** is a systematic method of constructing a portfolio using predefined signals or rules. Each allocation is defined by a set of portfolio weights (which can be positive or negative) applied on a specific day and held for one trading session. From one day to another, an allocation can rebalance its weights depending on the rules used to compute it.

1.2 Dataset Description

For each allocation and each timestamp, we are provided with:

- **20-day history** of allocation returns
- **20-day history** of volume-weighted liquidity behavior
- **Median daily turnover** of the allocation
- **Anonymized group** of the allocation
- **Target variable** : the true return of the allocation on the following trading day

The dataset consists of:

- **Training set** : 527,073 observations
- **Test set** : 31,870 observations

1.3 Problem Formulation

This is a **binary classification problem** where we must predict the *sign* of the allocation's next-day return:

- *Positive return* $\Rightarrow$ Trust the allocation (go long)
- *Negative return* $\Rightarrow$ Short the allocation

1.3.1 Mathematical Definition

For a given trading day t , an allocation S , and M assets in a trading universe, let:

- $\mathbf{w}_{S,t} = (w_{S,t,1}, w_{S,t,2}, \dots, w_{S,t,M})$ be the weights of allocation S at time t

- $r_{i,t+1}$ be the performance (return) of asset i from day t to day $t + 1$

The realized return of allocation S at time $t + 1$ is given by:

$$r_{S,t+1} = \sum_{i=1}^M w_{S,t,i} \cdot r_{i,t+1} \quad (1)$$

The task is to predict the sign of $r_{S,t+1}$ using the 20-day historical information available at time t .

1.3.2 Evaluation Metric

We evaluate the model based on **accuracy**, which measures how often the model correctly predicts the direction (sign) of the allocation's next-day return.

For each observation i indexed by timestamp t and allocation S , we compute:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}_{\{\text{sign}(\hat{r}_i) = \text{sign}(r_i)\}} \quad (2)$$

where:

- N is the total number of observations
- $\text{sign}(x) = 1$ if $x > 0$, else 0
- $r_i = r_{S,t+1}$ is the true next-day return for observation i
- $\hat{r}_i = \hat{r}_{S,t+1}$ is the predicted next-day return for observation i
- $\mathbf{1}_{\{\cdot\}}$ is the indicator function

Only the sign of the predicted return matters, not its magnitude.

2 Data Analysis

2.1 Missing Values Analysis

2.1.1 Overview

Missing values are a critical aspect of any machine learning project. They can arise from various sources such as data collection errors, merging issues, or legitimate absence of information. Understanding their distribution and impact is essential for building robust models.

2.1.2 Observations

Our analysis of the training dataset reveals:

- **Training set size:** 527,073 observations

- **Total features:** 44 columns (including identifiers and target)
- **Total data points:** $527,073 \times 44 \approx 23.2$ million values

A comprehensive scan of the dataset shows the distribution of missing values across all columns. The presence of NaN values in historical returns (RET_i), signed volumes ($SIGNED_VOLUME_i$), and other features can significantly impact model performance: Mostly on `signed_return_1` and `signed_return_20`, that looks odd

2.1.3 Why Missing Values Are Problematic

Missing values present several challenges:

1. Information Loss When data is missing, we lose potentially valuable information about an allocation's behavior. This reduces the effective feature space available for the model to learn patterns.

2. Biased Estimates Simple imputation strategies (such as replacing NaN with 0) introduce bias:

- Setting returns to 0 when data is missing suggests zero performance, which may not be accurate
- This can mislead models into learning incorrect relationships between features and targets
- For Ridge regression, this creates artificial zero-coefficients that may distort feature importance

3. Model-Specific Issues

- **Ridge Regression:** Assumes continuous input values. Replacing NaN with 0 without normalization can bias coefficients toward features with larger magnitudes
- **LightGBM:** While it handles missing values natively by learning optimal split points around them, the presence of many NaN values increases model complexity and computational cost

4. Generalization Problems If the test set has a different distribution of missing values than the training set, models may perform poorly on unseen data due to domain shift.

2.1.4 Benchmark's Approach

The benchmark notebook handles missing values using a simple strategy:

```
# Ridge: Replace NaN with 0
X_train[features].to_numpy(na_value=0)
X_test[features].fillna(0).to_numpy(na_value=0)

# LightGBM: Let the algorithm handle it natively
lgbm.Dataset(X_local_train, label=y_local_train.values)
```


While practical, this approach may not be optimal. Future improvements could include:

- Forward-fill or backward-fill strategies for time-series data
- Mean or median imputation per allocation
- K-NN imputation based on similar allocations
- Creating explicit “missing indicator” features

3 Conclusion